ПРАКТИЧЕСКОЕ РУКОВОДСТВО ПО DEVOPS

Современные инструменты CI/CD

Что такое CI и CD простыми словами?

Автоматический конвейер, избавляющий команду от рутинных ручных проверок и сборок

CI — Непрерывная интеграция

Разработчики часто отправляют код в общий репозиторий. Робот сразу собирает проект и запускает тесты.

Цель: Найти ошибки в коде за 3 минуты, а не перед релизом

Результат: В главной ветке всегда рабочий и проверенный код

CD — Доставка и развертывание

Автоматическая доставка готовой программы на тестовые серверы или прямо пользователям в прод.

Delivery: Сборка артефакта и ручной клик на выкатку

Deployment: Полностью автоматическая публикация без участия людей

Из чего состоит пайплайн: 4 главных понятия

Базовые термины, которые используются во всех современных CI/CD платформах

Триггер (Trigger)

Событие запуска: push в ветку, создание Pull Request или расписание по таймеру

Стадии и шаги

Логические этапы: проверка синтаксиса, компиляция, тесты и деплой

Раннер (Runner)

Изолированная машина или контейнер, где физически запускаются команды

Артефакты

Финальные файлы работы: скомпилированный бинарник, Docker-образ или отчет тестов

Квартет лидеров: разная философия работы

Каждый инструмент создавался под разные задачи и типы инфраструктуры



GitHub Actions

Модульный конструктор из готовых блоков маркетплейса



GitLab CI/CD

Монолитная комбайн-платформа «всё в одном окне»



Bitbucket

Максимально простой CI для пользователей Jira



Jenkins

Швейцарский нож с неограниченной свободой настройки

GitHub Actions: Концепция и плюсы

Как устроен самый быстрорастущий инструмент в открытом коде

Хранение рядом с кодом: Все файлы пайплайнов лежат в папке `.github/workflows/` в формате YAML.

Огромный Marketplace: Не нужно писать скрипты с нуля — миллионы разработчиков делятся готовыми шагами (деплой в AWS, уведомление в Telegram).

Облачные раннеры от GitHub: Чистые виртуальные машины Ubuntu, Windows и macOS поднимаются за секунды без поддержки своих серверов.

Реакция на любые события: Пайплайн можно запустить не только на коммит, но и на открытие issue, добавление комментария или релизный тег.

GitHub Actions: Пример пайплайна

Файл .github/workflows/test.yml с построчным объяснением логики

```yaml
name: CI Pipeline
on: [push, pull_request]

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - name: Скачать репозиторий
        uses: actions/checkout@v4

      - name: Установить Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20

      - name: Запуск тестов
        run: |
          npm ci
          npm test
```

1. Триггер on: [push, pull_request]

Сборка стартует автоматически при любой отправке изменений в ветку.

2. runs-on: ubuntu-latest

GitHub бесплатно выделяет свежую виртуальную машину с Linux.

3. Директива uses

Подключение проверенных модулей из официального каталога.

4. Директива run

Обычные консольные команды терминала для установки пакетов и тестов.

GitLab CI/CD: Концепция и плюсы

Истинный DevSecOps: полный жизненный цикл продукта в одной системе

Один конфигурационный файл: Вся схема описывается в корневом файле `.gitlab-ci.yml` со строгой структурой фаз.

Контейнеризация по умолчанию: Каждая джоба выполняется внутри указанного Docker-образа, обеспечивая идеальную изоляцию.

Безопасность из коробки: Встроенные сканеры уязвимостей в коде (SAST), зависимостях и образах контейнеров.

GitLab Runner: Легковесный агент на Go, который можно поставить на любой сервер или в Kubernetes за 5 минут.

GitLab CI/CD: Пример пайплайна

Файл .gitlab-ci.yml с разделением на этапы и сохранением артефактов

```yaml
image: node:20-alpine

stages:
  - test
  - build

test-job:
  stage: test
  script:
    - npm ci
    - npm test

build-job:
  stage: build
  script:
    - npm run build
  artifacts:
    paths:
      - dist/
```

1. image: node:20-alpine

Глобальный легкий Docker-контейнер для всех задач пайплайна.

2. stages: [test, build]

Строгая очередность: сборка начнется только при успехе тестов.

3. script: [...]

Список bash-команд, которые исполняются раннером внутри контейнера.

4. artifacts: paths

Папка dist/ архивируется и передается на следующие шаги или деплой.

Bitbucket Pipelines: Концепция и плюсы

Оптимизированное решение для команд, живущих в экосистеме Atlassian

Прозрачность в Jira: Номер задачи из названия коммита связывается с задачей: менеджер видит статус сборки прямо в карточке Jira.

Минимум настроек: Включается одним переключателем в веб-интерфейсе, не требует сложных инфраструктурных манипуляций.

Контейнеры для каждого шага: Каждый step запускается в своем Docker-образе с автоматическим кэшированием npm или pip.

Развертывания (Deployments): Встроенный учет окружений (Test, Staging, Production) с возможностью ручного подтверждения.

Bitbucket Pipelines: Пример пайплайна

Файл bitbucket-pipelines.yml с кэшированием и ручным подтверждением выкатки

```yaml
image: node:20

pipelines:
  branches:
    main:
      - step:
          name: Проверка и тесты
          caches:
            - node
          script:
            - npm ci
            - npm test

      - step:
          name: Деплой на Prod
          trigger: manual
          script:
            - ./deploy.sh
```

1. branches: main

Пайплайн запускается только при попадании кода в главную ветку main.

2. caches: [node]

Кэш node_modules сохраняется между сборками, ускоряя шаги в 3 раза.

3. trigger: manual

Шаг деплоя ждет нажатия кнопки ответственным инженером в UI.

4. script: ./deploy.sh

Запуск скрипта доставки на целевые боевые сервера.

Jenkins: Концепция и плюсы

Мощный инструмент с открытым исходным кодом для нестандартных задач

Полная независимость: Разворачивается на собственных физических серверах или в закрытом контуре без интернета.

1800+ плагинов: Можно состыковать с любой устаревшей базой данных, мейнфреймом или экзотическим языком программирования.

Язык Groovy (Jenkinsfile): В отличие от статического YAML, пайплайн — это полноценная программа с циклами, условиями и функциями.

Архитектура Controller / Agent: Центральный сервер только раздает команды, а тяжелую работу делают агенты.

Jenkins: Пример кода (Jenkinsfile)

Современный декларативный пайплайн (Declarative Pipeline)

```
pipeline {
  agent any

  stages {
    stage('Сборка и Тест') {
      steps {
        echo 'Запуск тестов...'
        sh 'npm ci'
        sh 'npm test'
      }
    }

    stage('Деплой') {
      steps {
        echo 'Выкатка на сервер...'
        sh './deploy.sh'
      }
    }
  }
}
```

1. pipeline { ... }

Обертка современного безопасного декларативного синтаксиса.

2. agent any

Запуск на любом свободном воркере (можно указать метку, например docker).

3. stage('...') { steps { ... } }

Наглядные визуальные блоки для отображения шкалы времени в UI.

4. Команда sh '...'

Выполнение shell-скрипта на агенте с передачей кода возврата.

Синтаксис: YAML против Groovy

В чем фундаментальная разница в написании конфигураций пайплайнов

Декларативный YAML (GitHub, GitLab, Bitbucket)

Простота чтения: Понятен любому разработчику без знания языков скриптов

Безопасность: Нельзя выполнить произвольный опасный код на сервере

Ограничение: Сложные условия и циклы требуют костылей или внешних скриптов

Императивный Groovy (Jenkins)

Абсолютная мощь: Полноценные циклы, парсинг JSON, вызовы API прямо в пайплайне

Сложный порог входа: Нужно изучать Groovy и специфику Jenkins Sandbox

Риск спагетти-кода: Пайплайн рискует превратиться в неподдерживаемый монолит

Цена владения: SaaS против Собственных серверов

80%

Времени DevOps уходит на инфраструктуру Jenkins, а не на пайплайны

Бесплатный софт (Open Source) обманчив: обновление версий Java, конфликты плагинов и администрирование дисков часто обходятся компании в разы дороже ежемесячной подписки на облачный сервис.

Итоговая сравнительная таблица

Прямое сопоставление ключевых характеристик

Критерий	GitHub Actions	GitLab CI/CD	Bitbucket	Jenkins
Синтаксис	YAML	YAML	YAML	Groovy DSL
Сложность старта	**Очень низкая**	**Средняя**	**Очень низкая**	**Высокая**
Хостинг серверов	SaaS + свои воркеры	SaaS или On-Prem	SaaS + свои воркеры	Только свой сервер
Готовые шаги	20 000+ Actions	Шаблоны CI/CD	Pipes каталог	1800+ плагинов
Главное преимущество	Маркетплейс	Безопасность и Git	Связка с Jira	Гибкость под On-Prem

Что выбрать для своего проекта?

Простые практические критерии для принятия решения в команде

GitHub Actions

Стартапы и Open Source

Если код уже на GitHub, запуск займет 10 минут без серверов и DevOps.

GitLab CI/CD

Компании и безопасность

Когда нужен строгий аудит, автосканирование уязвимостей и единый Git.

Bitbucket

Atlassian-стек

Когда все процессы завязаны на тикеты Jira и не требуются сложные сценарии.

Jenkins

Банки и закрытые сети

Когда данные строго запрещено отдавать в облака и есть отдельная команда DevOps.

ГЛАВНЫЙ ВЫВОД

Начинайте с простого

Лучший CI/CD инструмент — тот, который решает задачи вашей команды сегодня без лишних накладных расходов на поддержку